

Multimodal RAG - Complete Code Walkthrough

Code Walkthrough

Project	E:\Rag_App\multimodal-rag
Purpose	Every module, every function, and the full request flow
Stack	PyMuPDF, Gemini, Groq, Qdrant, FastAPI, Streamlit
Scale	2 documents, 457 indexed chunks, 176 figures
Cost	Free tier throughout



What this system does

It reads PDFs that contain text, diagrams, screenshots and tables, and answers questions about them by returning a **written answer plus the exact image files the answer depends on**. Not a description of a picture - the picture itself, byte-for-byte as it appeared in the source document.

The one idea everything else follows from

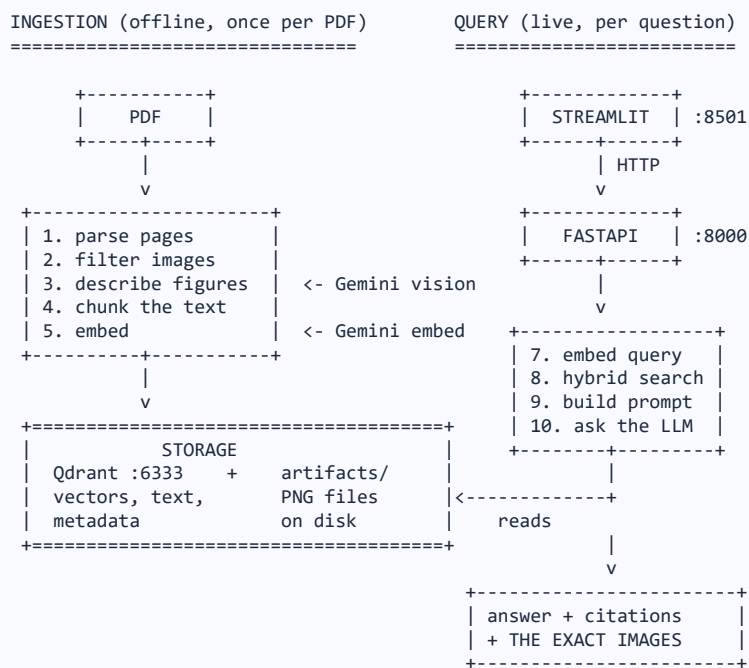
A picture cannot be searched. Search compares meaning between pieces of text, and a PNG contains no text. So during ingestion a vision model **writes text about every figure**, and that text is what gets indexed. At answer time the description is discarded and the real PNG is handed to the model. Descriptions get you **to** the image; the image itself is what gets read and returned.

Everything in the codebase is a consequence of that round trip. Enrichment exists to create the description. Retrieval searches descriptions. The prompt builder swaps descriptions back out for pixels. The response returns file paths.

The 60-second mental model

Two pipelines that never call each other. They communicate only through storage.

The whole system on one page



Ingestion is slow, expensive and runs once. Query is fast and runs constantly. Because they are decoupled, you can rebuild the index without re-parsing a PDF, and tune retrieval without spending any vision quota.

Directory map

multimodal-rag/	
├─ app/	the pipeline - all logic lives here
│ │ └─ config.py	every setting, read from .env once
│ │ └─ llm.py	the ONLY file that talks to a model provider
│ │ └─ models.py	data shapes: Chunk + the schemas models must fill
│ │ └─ enrich.py	vision calls: describe figures, transcribe pages
│ │ └─ ingest.py	PDF -> Chunks
│ │ └─ store.py	Chunks -> Qdrant
│ │ └─ retrieve.py	question -> ranked chunks
│ │ └─ answer.py	chunks -> answer + images
│ └─ diagram.py	"draw me a diagram" branch
├─ api.py	FastAPI - thin, no logic
├─ ui.py	Streamlit - thin, no logic
├─ run_ingest.py	CLI: PDF -> artifacts/
├─ run_index.py	CLI: artifacts/ -> Qdrant
├─ eval/	golden question set + scoring
├─ tests/smoke_test.py	end-to-end checks
├─ docs/	this document and the PDF renderer
├─ data/	your source PDFs
├─ artifacts/	extracted PNGs + manifest + vision cache
└─ qdrant_storage/	Qdrant's persistent volume

The rule that keeps it testable

`api.py` and `ui.py` contain no pipeline logic. They only call into `app/`. That is why the whole system can be driven from a Python prompt with no server running - which is exactly how every diagnostic in this project was done.

Module 1 - config.py

Holds every setting. Does no work. Reads `.env` once at import time, so any module can `from app import config` and read a value without repeating environment plumbing.

Setting	Value	Why it is that value
LLM_CHAIN	4 models	Tried top to bottom until one answers
EMBED_MODEL	gemini-embed-01	Free; no fallback allowed (see below)
EMBED_DIM	768	Model default is 3072; 768 is smaller, faster, and plenty here
GEMINI_SLEEP_SECONDS	4.5	~13 vision calls/min, under the free-tier limit
EMBED_BATCH_SIZE	16	Items per embedding call
EMBED_SLEEP_SECONDS	11	16 items every 11s = ~87/min, under the 100/min cap
MIN_IMAGE_PIXELS	10000	Smaller than this is decoration
MAX_FLAT_COLOUR_RATIO	0.95	95%+ one colour = a rule or a fill
PAGE_RENDER_DPI	200	Readable enough for a vision model to transcribe
CHUNK_WORDS	700	Chunk size
CHUNK_OVERLAP_WORDS	80	Stops an idea being cut in half at a boundary
TOP_TEXT	8	Text chunks sent to the model
TOP_IMAGES	3	Images sent to the model - the modality quota
CANDIDATES	40	Pulled from each retriever before fusing
ANSWER_IMAGE_MAX_PX	1280	Downscale before attaching, to save tokens

The chain

```
LLM_CHAIN = [  
    # (provider, model, can_see_images)  
    ("gemini", "gemini-3.5-flash", True),  
    ("gemini", "gemini-flash-lite-latest", True),  
    ("groq", "llama-3.3-70b-versatile", False),  
    ("groq", "llama-3.1-8b-instant", False),  
]
```

The third element is the important one. Groq has no vision model, so those two entries are marked text-only. `ask()` reads that flag and simply does not attach images for them - which works because of the prompt rule described in Module 7.

The two Gemini entries have **separate free-tier quotas**, which is why the second is a genuine fallback and not a duplicate. This was proven in practice: when a 165-image ingest exhausted the primary's daily quota, the second Gemini model carried every subsequent request.

Why embeddings have no fallback

Vectors from two different models are not comparable. If documents were embedded with one model and a query with another, search returns nonsense **silently** - no error, just wrong results. So `embed()` retries and then fails loudly. It never switches model. Changing the embedding model means re-indexing everything, and that must be a deliberate act.

Module 2 - llm.py

The only file that talks to a model provider. Everything else calls `ask()`, `ask_json()` or `embed()` and never knows which model answered.

Function	Purpose
<code>ask(prompt, images)</code>	Free-text answer. Returns <code>(text, "provider:model")</code>
<code>ask_json(prompt, schema, images, needs_vision)</code>	Structured answer validated against a pydantic schema. Returns <code>(dict, "provider:model")</code>
<code>embed(texts, task_type)</code>	List of strings to list of vectors
<code>embed_query(text)</code>	One query to one vector, using the query task type
<code>_retry_after(err, fallback)</code>	Reads the server's own retry advice out of a 429
<code>_normalise(vector)</code>	Scales a vector to unit length

ask() - the fallback chain

```
def ask(prompt, images=None):
    failures = []
    for provider, model, can_see in config.LLM_CHAIN:
        try:
            text = CALLERS[provider](model, prompt, images if can_see else None)
            return text, f"{provider}:{model}"
        except Exception as err:
            failures.append(...)
    raise RuntimeError("Every provider failed")
```

Note `images if can_see else None`. A text-only model is not skipped - it is called **without** the pictures. It still produces a grounded answer because the prompt already contains every figure's written description.

ask_json() - structured output

Same loop, but each provider is asked for JSON matching a schema.

- **Gemini** takes `response_schema=YourPydanticModel` and enforces it.
- **Groq** takes `response_format={"type": "json_object"}`, which guarantees

valid JSON but **not** that it matches your schema.

`needs_vision=True` skips text-only models entirely. Enrichment uses this: a model that cannot see the image is useless there, not merely degraded.

A schema is only a contract where the provider enforces it

Because Groq does not enforce the schema, the Groq path sometimes returned an answer with `used_pages` missing. `answer.py` backfills it. Treat schema compliance as best-effort on any provider whose JSON mode is not schema-aware, and validate on your side.

embed() and the 429 that taught us something

```
def _retry_after(err, fallback):
    match = _RETRY_HINT.search(str(err))    # "Please retry in 34.5s"
    if match:
        return min(float(match.group(1)) + 2.0, 120.0)
    return fallback
```

The original code used plain exponential backoff: 1, 2, 4, 8 seconds - 15 in total. The server was explicitly asking for **34.5 seconds**. Every attempt was burned before the quota window reopened, and indexing failed.

The lesson generalises: when a server tells you how long to wait, waiting that long beats any backoff curve you invent.

Module 3 - models.py

The data shapes. Two categories: what we store, and what models must return.

Chunk - the single record type

```
@dataclass
class Chunk:
    id: str                # "aws-notes:p42:i2"
    doc_id: str
    page: int
    kind: str              # "text" | "image"
    text: str              # the searchable surface
    heading: str = ""      # section trail or figure caption
    image_path: str | None = None
```

Why there is only one record type

A paragraph, a diagram and a table are all a **Chunk**. They differ only by **kind** and whether they carry an **image_path**.

One type means one collection, one search path, and both modalities ranked against each other in a single query. Two types would have meant two searches and an arbitrary rule for merging them.

The **id** format **docid:pN:tN** or **docid:pN:iN** is human-readable on purpose - when a result looks wrong, the id alone tells you the document, page and kind.

Schemas the model must fill in

Schema	Used by	Fields
PageTranscript	vision route	markdown, page_summary, figures[]
Figure	inside PageTranscript	caption, description, kind
ImageDescription	text route	caption, description, extracted_text, kind
AnswerPayload	answering	answer, used_image_ids[], used_pages[]

These are pydantic models passed straight to Gemini as **response_schema**. The **Field(description=...)** text is not decoration - it is sent to the model and tells it what to put in each field.

AnswerPayload.used_image_ids is what makes **cited_by_model** possible: the model reports which figures it actually relied on, so the UI can separate "used in the answer" from "also retrieved".

Module 4 - enrich.py

Where pictures become searchable. Two jobs and a cache.

Function	Purpose
Cache	A JSON file so a re-run never repeats a vision call
<code>transcribe_page(png, page_no, cache)</code>	Whole page to markdown + figure list
<code>describe_image(png, context, key, cache)</code>	One figure to a description
<code>_throttle()</code>	Sleeps <code>GEMINI_SLEEP_SECONDS</code> between calls

The cache is what makes iteration affordable

```
class Cache:
    def get(self, key): ...
    def set(self, key, value):
        self.data[key] = value
        self.path.write_text(json.dumps(self.data, indent=2))
```

Written to `artifacts/<doc_id>/enriched.json` after **every** call, not at the end. That matters: when the 69-page AWS ingest was interrupted, all 165 vision results were already on disk. Re-running was nearly instant.

Note the order inside `describe_image`: a cache hit returns **before** `_throttle()`. So a re-run does not sleep 4.5 seconds per cached image.

Why surrounding page text is passed in

```
DESCRIBE_PROMPT = """Describe this figure...
Text surrounding this figure on the page (use it to resolve references like
"Figure 4", but describe only the image itself):
---
{context}
---"""
```

Without context, a diagram captioned only "Figure 4" is meaningless. With the page text, the model knows Figure 4 is the auth flow and can say so.

extracted_text is the quiet hero

`ImageDescription.extracted_text` captures every word **inside** the picture. That is how a search for `issue_refund` finds a code screenshot: the string exists only as pixels, but after enrichment it exists as text in the index, and BM25 can match it literally.

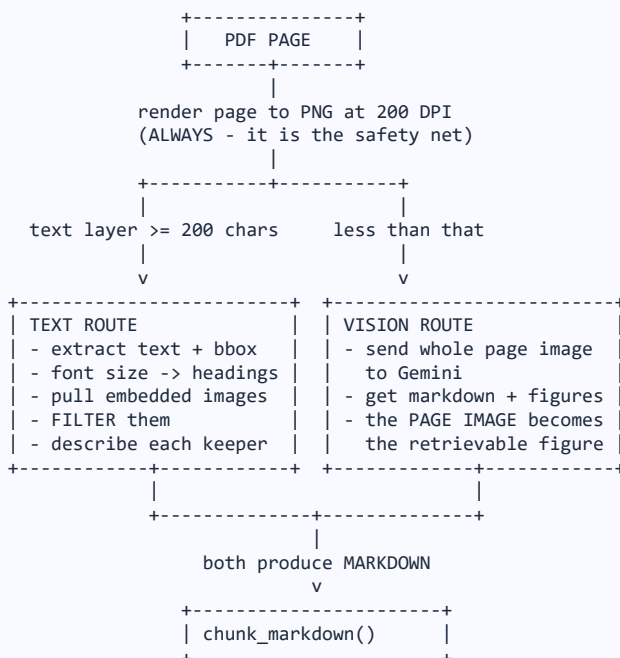
Module 5 - ingest.py

PDF in, **Chunk** list out. The largest module, because PDFs are messy.

Function	Purpose
<code>slugify(name)</code>	Filename to <code>doc_id</code>
<code>flat_colour_ratio(png)</code>	Fraction of pixels that are the single most common colour
<code>page_to_markdown(page)</code>	Text + font sizes to markdown with <code>##</code> headings
<code>tidy_markdown(md)</code>	Repairs transcriptions returned without line breaks
<code>chunk_markdown(md)</code>	Split on headings, then window long sections
<code>figure_to_text(figure)</code>	Flatten a description into the searchable string
<code>ingest_pdf(path)</code>	The entry point - loops pages, picks a route
<code>_ingest_text_page(...)</code>	Route A
<code>_ingest_vision_page(...)</code>	Route B
<code>_write_manifest(...)</code>	Save everything to <code>manifest.json</code>

Route selection

Every page takes one of two routes



Your two documents took different routes, which is why both paths are proven: `operating-ai-agents` was 11 vision pages; `aws-notes` was 64 text pages and 5 vision pages.

Why every page is rendered, even text pages

Some diagrams are not stored as images at all - they are vector drawing instructions, so `page.get_images()` finds nothing. The page render always contains them. It costs one PNG per page and eliminates an entire class of silent failure.

page_to_markdown() - headings from font size

```
sizes = [span["size"] for block in text_blocks
          for line in block["lines"]
          for span in line["spans"]]
body_size = statistics.median(sizes)
...
is_heading = biggest >= body_size * 1.15 and len(text) < 80
```

The **median** font size is body text. Anything 15% larger and short is a heading. This converts a text page into the same markdown shape the vision route produces - so one chunker handles both routes.

The image filters

```
if pix.width * pix.height < config.MIN_IMAGE_PIXELS: continue
if flat_colour_ratio(png) > config.MAX_FLAT_COLOUR_RATIO: continue
```

`flat_colour_ratio` thumbnails to 64x64 first, so the check costs nothing.

A known weakness, visible in your own data

These filters catch rules, fills and tiny icons. They do **not** catch a colourful logo. The AWS PDF yielded 165 "figures" from 69 pages because a branded header repeats on every page. Each one cost a vision call, and collectively they exhausted the primary model's daily quota. A content hash that skips any PNG already seen in the same document would fix this in about five lines.

chunk_markdown() - split on meaning, then on size

```
for line in markdown.splitlines():
    if line.lstrip().startswith("#"):
        sections.append((heading, buffer)); heading = line.lstrip("#").strip()
...
for head, body in sections:
    for start in range(0, len(words), step):
        chunks.append((head, " ".join(words[start:start + max_words])))
```

Headings first, size second. Cutting every 700 words regardless of structure would split an explanation across two chunks, leaving neither able to answer anything. Each chunk also carries its heading, which helps both retrieval and the final answer.

tidy_markdown() - a real bug, fixed defensively

Vision models sometimes return an entire page as one long line, which hides every heading from the chunker and collapses the page into a single blob.

```
markdown = re.sub(r"(?!\\n)(#{1,6} [A-Z0-9])", r"\\n\\n\\1", markdown)
```

The `[A-Z0-9]` guard matters: without it, a `# fast, cheap` comment inside a code block would be treated as a heading. Requiring an uppercase letter or digit after the hash avoids that.

Module 6 - store.py

Everything that touches Qdrant.

Function	Purpose
<code>point_id(chunk_id)</code>	Readable string id to a stable UUID
<code>ensure_collection(recreate)</code>	Create the collection and payload indexes
<code>count()</code>	Exact point count
<code>index_chunks(chunks)</code>	Embed and upsert, paced for the free tier
<code>load_manifest(path) / load_all_manifests()</code>	Read chunks back from disk
<code>all_payloads()</code>	Scroll everything - used to build BM25
<code>dense_search(vector, limit, kind)</code>	Vector similarity with optional filter

point_id() - why the hash

Qdrant point ids must be an integer or a UUID, but our ids are readable strings like `aws-notes:p42:i2`. A UUID5 hash gives a **deterministic** id:

```
_NAMESPACE = uuid.UUID("6f1c9d0e-...")
def point_id(chunk_id):
    return str(uuid.uuid5(_NAMESPACE, chunk_id))
```

Deterministic means re-indexing **overwrites** rather than duplicating. That is why `run_index.py` is safe to re-run - which mattered when indexing had to be restarted after an interruption.

ensure_collection() - the payload indexes are not optional

```
for field in ("kind", "doc_id"):
    _client.create_payload_index(..., field_schema=qm.PayloadSchemaType.KEYWORD)
```

Indexing `kind` is what makes the modality quota a fast server-side filter rather than a Python loop over every result.

index_chunks() - paced deliberately

```
for start in range(0, total, batch_size):
    vectors = embed([c.text for c in batch])
    _client.upsert(...)
    if done < total and pause:
        time.sleep(pause)
```

The lesson that cost the most time

The free tier allows ~100 embedded items per minute, and **each text inside a batch counts individually**. A batch of 16 is sixteen requests, not one. 457 chunks pushed at full speed hit a 429 within seconds. Sleeping 11s between batches is far cheaper than retrying after the fact.

Module 7 - retrieve.py

Question in, ranked chunks out.

Function	Purpose
<code>_tokenise(text)</code>	Lowercase, split on non-alphanumerics, drop 1-char tokens
<code>load_index(force)</code>	Build the BM25 index; rebuilds itself when stale
<code>bm25_search(query, limit)</code>	Keyword ranking
<code>reciprocal_rank_fusion(*lists)</code>	Merge ranked lists by position
<code>search(query, ...)</code>	The public entry point

Why two searches

	Dense (Qdrant)	Sparse (BM25)
Matches on	meaning	exact strings
Wins at	"renew credentials" ~ "refresh token"	<code>issue_refund</code> , Figure 4.2
Fails at	exact identifiers it blurs together	any paraphrase

Measured on the golden set: dense scored hit@5 of 97.5%, BM25 92.5%, and the two fused reached **100%**. Neither alone was enough.

load_index() - a real bug and its fix

The BM25 index lives in process memory and was originally built once. When a new document was indexed by another process, the running API kept using the old index - so the new document was findable by meaning but **invisible** to keyword search. A silent half-failure, very hard to spot from outside.

```
if _bm25 is not None and not force:
    if store.count() == len(_payloads):
        return
    print("[retrieve] collection changed, rebuilding BM25")
```

One cheap count against Qdrant per search, and the whole class of problem disappears.

reciprocal_rank_fusion() - merge by rank, not score

```
for rank, (payload, _score) in enumerate(results):
    entry["score"] += 1.0 / (RRF_K + rank + 1)    # RRF_K = 60
```

Why not just add the scores

A Qdrant cosine score and a BM25 score are on completely different scales. Adding them means whichever happens to produce bigger numbers wins every time. Ranks are comparable across any two rankers. An item near the top of both lists beats an item only one liked.

The modality quota

```
texts = [e for e in fused if e["payload"]["kind"] == "text"][:top_text]
images = [e for e in fused if e["payload"]["kind"] == "image"][:top_images]
```

Taken **separately**, not as one top-k list.

Without this, the project does not work

Text chunks win on raw score almost every time. A flat top-8 would return eight paragraphs and zero figures, and the entire point of the system - returning the exact image - would never fire.

found_by - the debugging handle

Every hit records whether **dense**, **bm25**, or both surfaced it. When a result looks wrong, that single field tells you immediately whether the problem is in the embeddings or the keyword index.

Module 8 - answer.py

Where the round trip completes.

Function	Purpose
<code>_load_image(path, max_px)</code>	Read a PNG and downscale it
<code>build_prompt(question, texts, images)</code>	Assemble the prompt; returns <code>(text, label_map)</code>
<code>answer_question(question, ...)</code>	The full query path

The rule that shapes everything

The prompt must stand alone without the images

Every figure's written description is included in the prompt `text`. The real PNGs are attached as a bonus for models that can see. So when the chain falls through to a text-only model, the answer is still grounded - it reads descriptions instead of pixels. Quality drops; nothing breaks. And the user gets the same exact image files either way, because retrieval already chose them.

build_prompt() and the label map

Figures are labelled `IMAGE-1`, `IMAGE-2` in the prompt, and `label_map` records which retrieval hit each label refers to. When the model returns `used_image_ids: ["IMAGE-1"]`, that maps back to a specific chunk, page and file path. Short labels are far easier for a model to reference reliably than an id like `aws-notes:p42:i2`.

_load_image() - why downscale

A 200 DPI page render is roughly 1700x2200. Sending that raw costs a great deal of quota for no readability gain, so images are resized to fit `ANSWER_IMAGE_MAX_PX` (1280) with LANCZOS - big enough that labels inside a diagram stay legible.

answer_question() - the full path

```
texts, images = retrieve.search(question, ...)          # 1. retrieve
if allow_diagram and diagram.wants_diagram(question): # 2. drawing branch
    drawing = diagram.generate(question, texts)
prompt, label_map = build_prompt(question, texts, images) # 3. assemble
image_parts = [_load_image(p) for each image]          # 4. load REAL pixels
data, model_used = ask_json(prompt, AnswerPayload, images=image_parts) # 5. ask
```

Step 4 is the reversal. During search an image was represented by its description. Here that description is set aside and the actual PNG is loaded from disk, so the model can read detail the caption never mentioned.

Proof from your own run: asked about RDS, the answer listed the field names `ID`, `Name`, `Age`, `Salary` and the row `Adam, 34, 13000`. Those values exist nowhere as text in the index - the model read them off the pixels.

The used_pages backfill


```
used_pages = set(data.get("used_pages") or [])
if not used_pages:
    used_pages = {hit["payload"]["page"]
                  for label, hit in label_map.items() if label in cited}
```

Groq's JSON mode does not enforce the schema, so `used_pages` sometimes came back empty. It is backfilled from whichever figures the model **did** cite - that page genuinely is a source, so this is inference, not invention.

The response

Field	Meaning
<code>answer</code>	Markdown answer with inline page citations
<code>model</code>	Which model in the chain actually answered
<code>saw_images</code>	Whether the answering model could see pictures
<code>used_pages</code>	Pages the answer draws on
<code>images[]</code>	Every retrieved figure, each flagged <code>cited_by_model</code>
<code>text_sources[]</code>	Every text chunk with score and <code>found_by</code>
<code>diagram</code>	A generated diagram, or <code>None</code>
<code>intent</code>	"find" or "draw"

Everything retrieved is returned, not only what was cited - so you can see the near-misses. `cited_by_model` marks what the answer actually leaned on.

Module 9 - diagram.py

The "create an image" branch, done the free way: the model writes **Graphviz DOT source** and the browser renders it.

Function	Purpose
<code>wants_diagram(question)</code>	Keyword pre-filter, then an LLM classify for maybes
<code>looks_like_dot(source)</code>	Cheap structural validation
<code>clean_dot(source)</code>	Strip markdown fences the model sometimes adds
<code>generate(question, text_hits)</code>	Build the diagram, with one retry

Why source code instead of a text-to-image model

	Diagram-as-code (built)	Text-to-image (not built)
Cost	Free - it is text generation	Billed on every provider checked
Faithfulness	Labels are exact strings	The model paints text; it can garble or invent
Editable	Yes - fix one node	No - regenerate the whole picture
Install	None; Streamlit renders DOT natively	None, but needs billing
Scope	Flows, hierarchies, sequences	Anything visual

The two-stage intent check

```
DRAW_HINTS = re.compile(r"\b(draw|sketch|diagram|flow ?chart|visuali[sz]e|...)\b")

def wants_diagram(question):
    if not DRAW_HINTS.search(question):
        return False # no API call at all
    data, _ = ask_json(CLASSIFY_PROMPT.format(question=question), schema=Intent)
    return data.get("intent") == "draw"
```

An ordinary question pays **nothing** for intent detection. Only wording that looks like a drawing request costs one cheap classification call. Asking "show me the auth diagram" (a lookup) versus "draw the auth flow" (a request to create) is exactly the distinction the LLM stage resolves.

Generated output is always labelled

Anything drawn carries `origin="generated"` and the UI marks it clearly. A retrieved image is evidence from the document; a generated diagram is the model's assertion. Letting those blur would quietly destroy the guarantee the whole system exists to provide.

Module 10 - api.py

FastAPI. Thin on purpose - every endpoint is a few lines that call into `app/`.

Endpoint	Purpose
GET /health	Qdrant reachability, point count, model chain, embed model
GET /api/v1/documents	What has been ingested, read off the artifact folders
POST /api/v1/query	The main path - calls <code>answer_question()</code>
POST /api/v1/ingest	Upload a PDF; runs in the background, returns a job id
GET /api/v1/jobs/{id}	Job status with page progress
GET /artifacts/...	Serves the PNG files (StaticFiles mount)

to_url() - disk path to browser URL

```
relative = Path(image_path).resolve().relative_to(config.ARTIFACT_DIR.resolve())
return "/artifacts/" + relative.as_posix()
```

The payload stores an absolute Windows path. The browser needs a URL. This is the only place that conversion happens, and `relative_to` raising on a path outside the artifact directory is a useful accident - it refuses to serve anything from elsewhere on disk.

Background ingest

```
JOBS = {}      # in-memory; fine for one user

def _run_ingest(job_id, pdf_path):
    JOBS[job_id]["status"] = "parsing"
    result = ingest_pdf(pdf_path)
    JOBS[job_id]["status"] = "indexing"
    store.index_chunks(result.chunks, verbose=False)
    retrieve.load_index(force=True)          # <- without this, BM25 stays stale
    JOBS[job_id].update(status="done", ...)
```

Ingestion takes minutes, so it runs in a background task and the caller polls.

`_pages_done()` counts rendered PNGs on disk to report real progress. A 69-page document can take half an hour on the free tier, and a status that just says "parsing" the whole time is indistinguishable from a hang.

Do not edit files while a background job runs under `--reload`

Uvicorn's reloader restarts the process on any file change, which kills the job mid-flight. This happened during development and cut an indexing run in half. Run long ingests from the CLI (`py run_ingest.py`) where nothing can restart the process.

Module 11 - ui.py

Streamlit. Also thin - it talks to the API over HTTP and renders what comes back. No retrieval or model logic, so either side can change independently.

Section	What it shows
Sidebar - health	"API up - N chunks indexed"
Sidebar - model chain	The four models and the embedding model
Sidebar - documents	Each doc with pages, chunks, images
Sidebar - sliders	<code>top_text</code> , <code>top_images</code> - live retrieval tuning
Sidebar - toggle	*Send images to the model* - simulates the text-only fallback
Sidebar - uploader	Drag a PDF in; shows job progress
Main - answer	Markdown, then a caption naming the model that answered
Main - diagram	Rendered Graphviz + a GENERATED warning + <code>.dot</code> download
Main - figures	Cited images inline; others in a collapsed expander
Main - retrieval detail	Every source with <code>found_by</code> and RRF score

The **Send images to the model** toggle is worth knowing about: it is how you verify that the text-only fallback still answers sensibly, without waiting for a real outage.

End-to-end trace

One question, every function that runs, in order.

Question: `"Explain me about RDS Relational Database Example with image diagram"`

Function-by-function

```
ui.py
  requests.post("/api/v1/query", {...})
  |
api.py :: query()
  |
answer.py :: answer_question()
  |
  +--> retrieve.py :: search()
  |   +--> llm.py :: embed_query()      question -> 768 numbers
  |   +--> store.py :: dense_search()   Qdrant, ~40 candidates
  |   +--> retrieve.py :: load_index()   rebuilds BM25 if stale
  |   +--> retrieve.py :: bm25_search()  keyword, ~40 candidates
  |   +--> reciprocal_rank_fusion()     merge by rank
  |   +--> modality_quota              8 text + 3 images
  |
  +--> diagram.py :: wants_diagram()    regex says no -> no API call
  +--> answer.py :: build_prompt()      text + DESCRIPTIONS
  |                                   + label_map IMAGE-1..3
  +--> answer.py :: _load_image() x3    REAL PNG bytes, downscaled
  |
  +--> llm.py :: ask_json()
  |   gemini-3.5-flash ---> 503 -----+
  |   gemini-flash-lite <-----+      served it
  |
  +--> parse used_image_ids -> cited
  +--> backfill used_pages if empty
  |
  v
api.py :: to_url()          disk path -> /artifacts/aws-notes/page_042_img_01.png
  |
ui.py :: st.image(...)      the browser fetches the PNG from FastAPI
```

Result:

```
model      : gemini:gemini-3.5-flash
pages      : [42]
IMAGE-1    page 42  cited=True   "Relational Database Example"
IMAGE-2    page 45  cited=False  "AWS Region with Availability Zones for RDS Multi-AZ"
IMAGE-3    page 11  cited=False  "AWS Architecture Diagram with ELB"
```

The answer described the diagram's own contents - the fields **ID**, **Name**, **Age**, **Salary**, and the row **Adam**, **34**, **13000**. None of that is text in the index. The model read it off the pixels of the file retrieval had chosen.

Free-tier engineering

Running entirely free is not just picking free models. Four mechanisms make it actually work.

Mechanism	Where	What it prevents
Vision throttle	<code>enrich._throttle()</code>	429 during ingestion
Embedding pacing	<code>store.index_chunks()</code>	429 during indexing
Server-advised retry	<code>llm._retry_after()</code>	Giving up before the window reopens
Result cache	<code>enrich.Cache</code>	Ever paying twice for the same picture

The limits that actually bite

Limit	Value	Consequence
Vision requests/day	exhausted by ~165 calls	Primary model dies for the day; chain covers it
Embed items/minute	100	Each text in a batch counts separately
Model availability	intermittent 503	Even a healthy model flaps under load

What each of these cost in practice

Ingesting a 69-page document produced 165 figures, each needing a vision call. That single ingest exhausted the primary model's daily quota - which is why tightening the image filters is the highest-value remaining optimisation. Fewer logos means fewer calls means quota left for actual questions.

Troubleshooting

Symptom	Cause	Fix
"does not contain information" for content you know exists	Not indexed yet	Check <code>/health</code> point count against the manifest
Sidebar count lower than the document's chunk count	Indexing interrupted	<code>py run_index.py</code> - safe to re-run
New document found by meaning but not by exact terms	Stale BM25	Fixed - <code>load_index()</code> self-heals; check the log line
429 RESOURCE_EXHAUSTED on generate	Daily vision quota gone	Wait for reset, or rely on the fallback
429 on embed with "retry in Ns"	Over 100 items/min	Raise <code>EMBED_SLEEP_SECONDS</code>
503 UNAVAILABLE	Model busy	Transient; chain absorbs it
Answers work, images 404	Artifacts moved	Re-run <code>run_ingest.py</code>
Ingest dies halfway	<code>--reload</code> restarted the process	Use the CLI for long ingests

Where to change what

You want to...	Edit
Swap or reorder models	<code>config.LLM_CHAIN</code>
Change chunk size	<code>config.CHUNK_WORDS</code>
Return more or fewer images	<code>config.TOP_IMAGES</code>
Make image filtering stricter	<code>config.MIN_IMAGE_PIXELS, MAX_FLAT_COLOUR_RATIO</code>
Change how figures are described	<code>enrich.DESCRIBE_PROMPT</code>
Change answer style	<code>answer.SYSTEM_RULES</code>
Change diagram style	<code>diagram.DIAGRAM_PROMPT</code>
Go faster and risk 429s	<code>config.EMBED_SLEEP_SECONDS, GEMINI_SLEEP_SECONDS</code>

The rule that matters most

Extraction and description quality cap everything downstream. No amount of retrieval tuning rescues a badly filtered, badly captioned index. When results disappoint, look in `artifacts/` and `enriched.json` **before** touching the search code.